

POC ORM



PRISMA

53742 - Nicolás Bolzico

53952 - Martín Cabrera

52285 - Agustín Gregoret



DRIZZLE

54326- Roque Doino

54689- Juan Pablo Villa

Introducción a ORM

Desfase Objeto-Relacional (Object-Relational Impedance Mismatch).

Aplicación

(TypeScript / JavaScript)

Modela en memoria con estructura orientada a objetos:

- **Objetos, Clases y Propiedades**

vs

Persistencia

(Bases de Datos Relacionales)

Almacena datos de forma relacional y tabular:

- **Tablas, Filas y Columnas**
- **Claves Primarias y Foráneas**

Introducción a ORM

Desfase Objeto-Relacional (Object-Relational Impedance Mismatch).

Aplicación

(TypeScript / JavaScript)

Modela en memoria con estructura orientada a objetos:

- **Objetos, Clases y Propiedades**

vs

Persistencia

(Bases de Datos Relacionales)

Almacena datos de forma relacional y tabular:

- **Tablas, Filas y Columnas**
- **Claves Primarias y Foráneas**

Mapeo Manual

Conversión Bidireccional Constante

Transformación manual de filas a objetos y de objetos a sentencias SQL **INSERT**, **UPDATE**, **DELETE**.

Ineficiencia de código y Riesgos en tiempo de ejecución

Alta sobrecarga de código repetitivo y elevado riesgo de errores e inconsistencias en tiempo de ejecución.

Introducción a ORM



Evolución e Historia de los ORM

Orígenes y Consolidación

Década de 1990

TopLink

Primer ORM (Smalltalk)

Principios de los 2000

Hibernate

Estándar Java

Mediados de los 2000

Django ORM / SQLAlchemy

Python

Type-Safety

Finales de los 2000

Entity Framework

.NET / C#

Mediados de la década de 2010

TypeORM

TypeScript / JavaScript

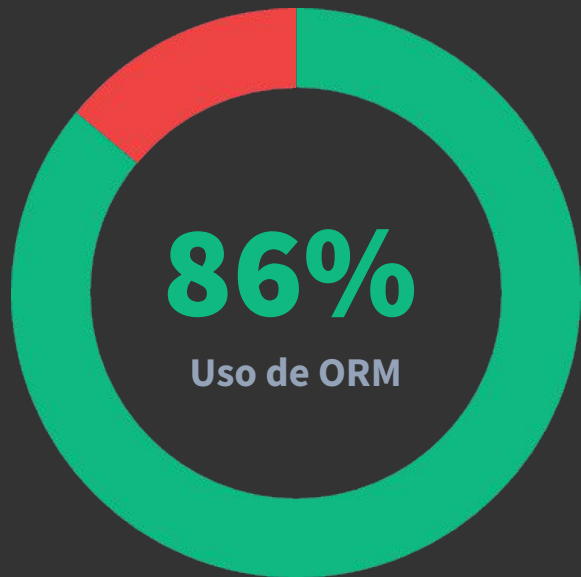
Últimos años

Prisma / Drizzle

TypeScript / JavaScript

Adopción de los ORM

Fuente: Python Developers Survey 2024 (+30,000 desarrolladores web)



- 86% — Utiliza ORM / Frameworks
- 14% — Solo Raw SQL (Sin ORM)

Organización de la Presentación

**1. Presentación
de Tecnologías**

**2. Factores No
Técnicos**

**3. Factores
Técnicos**

**4. Mercado
Laboral**

**5. Implementación
y demostración
práctica (PoC)**

6. Conclusiones

Presentación de cada Tecnología

DRIZZLE

¿Cuándo se creó y por qué?



FINALES DE 2022

Se creó en finales de 2022



CRECIMIENTO Y POPULARIDAD

Su crecimiento y popularidad se dio durante 2023-2024



OBJETIVO CLAVE

Drizzle se creó para bajar el peso de los ORMs



TRANSICIÓN AL SECTOR PRIVADO CON PLANETSCALE

Drizzle se integró y colaboró con PlanetScale para ofrecer soporte avanzado y optimizaciones específicas para su infraestructura de base de datos serverless. En 2026, PlanetScale adquirió Drizzle.

99

“If you know SQL, you know Drizzle”

FEATURES

- Type Safety

Usa TypeScript para detectar errores durante el desarrollo y ofrecer autocompletado.

FEATURES

- Consultas similares a SQL

Drizzle tiene una sintaxis muy parecida a SQL, esto facilita el aprendizaje para quienes conocen este lenguaje.

FEATURES

- Compatibilidad con múltiples bases de datos

PostgreSQL, MySQL, SQLite, SingleStore, entre otras.

FEATURES

- Liviano

Ocupa aproximadamente 7,4 KB comprimido y no posee dependencias externas.

FEATURES

- Serverless Ready

Cloudflare Workers, Vercel Edge, Bun y Deno

FEATURES

- Herramientas

Drizzle Kit (Para generar y aplicar migraciones)
y
Drizzle Studio (Interfaz gráfica)

FEATURES

- Compatibilidad

JavaScript

TypeScript



PRISMA

2016

Creación del
proyecto original de
Prisma

2020

Giro de negocio y
cambio estratégico
de enfoque

2021

Lanzamiento de la
primera versión
estable oficial

2025

Migración completa
del núcleo a
TypeScript puro

Fuertemente aceptado en la comunidad global de desarrollo



FEATURES

1. Modelo Declarativo.
2. Prisma Client (Type Safety / Autocompletado).
3. Prisma Migrate.
4. Manejo Intuitivo de Relaciones.
5. Compatibilidad Multi-Base de Datos.
6. Mapeador objeto-relacional moderno de código abierto para Node.js.



Generación automática de tipos TypeScript

1. `schema.prisma`
2. Creación del cliente a medida.
3. Interacción segura.
4. Prevención de bugs.
5. Velocidad al programar.

Factores No Técnicos

COMUNIDAD

- Drizzle ORM tiene más de 36000 estrellas en GitHub
- Más de 1700 forks
- Servidor de discord con 12.000 miembros
- ~ 76 M descargas/mes npm



Fork

1.7k



Star

36k



Dan (bloberenober) te invitó a unirte

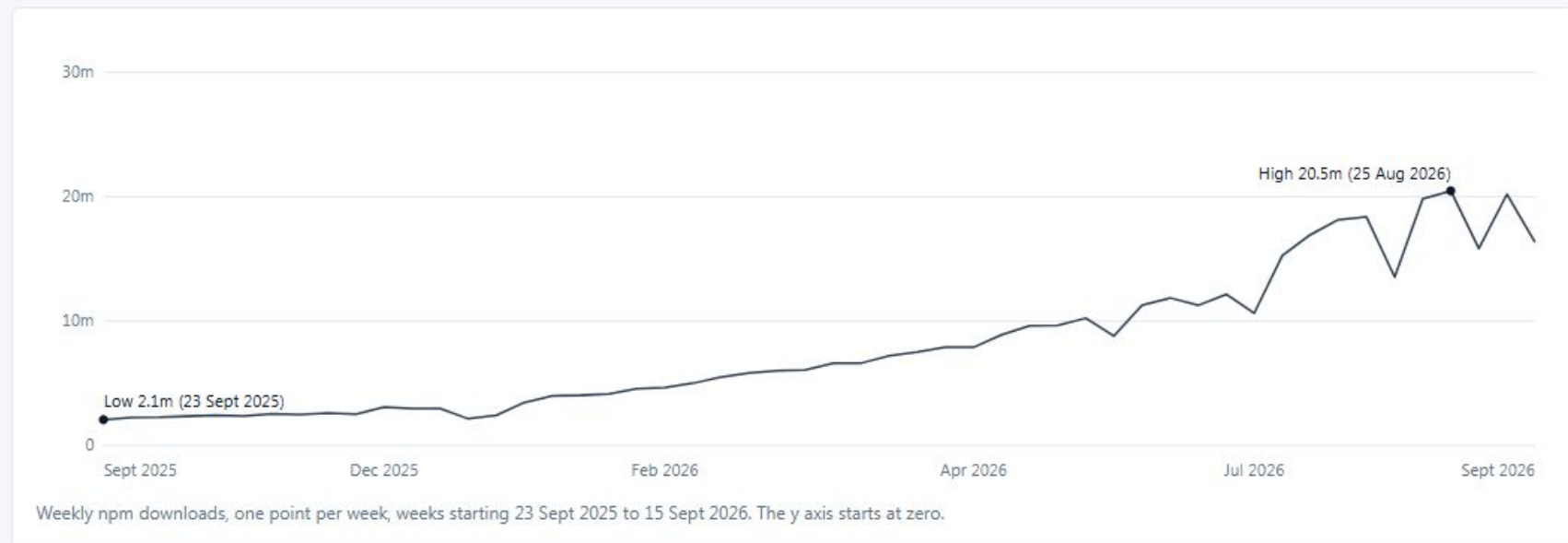
Drizzle Team

● 1,691 en línea ● 11,955 miembros

DESCARGAS NPM

Downloads

Last week 2026-09-15 to 2026-09-21



16,294,656 downloads last week, 75,943,842 in the last month. The 12-month change of +710% compares the average of the last 4 weeks with the first 4 weeks of the series.

RESOLUCIÓN DE BUGS

El equipo de Drizzle mantiene un desarrollo abierto (**Open Source**) publicado bajo la licencia Apache-2.0.

Entonces cuando los usuarios encuentran errores pueden reportarlos por los Issues de GitHub, haciendo Pull Request o escribiéndole al equipo por el discord oficial.

El proyecto se actualiza **frecuentemente** con nuevas versiones corrigiendo errores y mejorando el rendimiento. Haciendo mucho énfasis a lo que dice la comunidad, teniéndolos muy en cuenta.

CURVA DE APRENDIZAJE

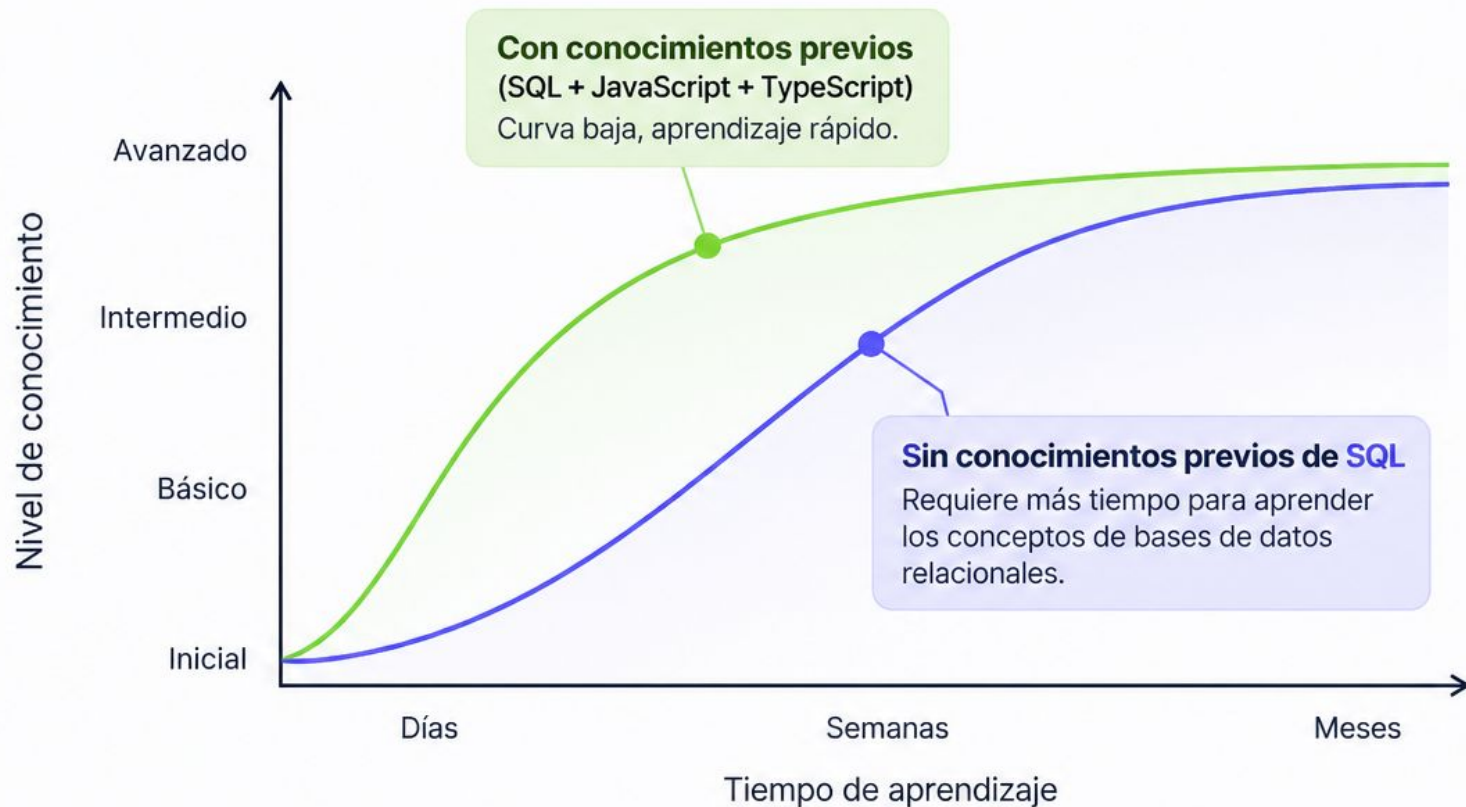
Textualmente la documentación de Drizzle dice lo siguiente:

"zero to no learning curve"

CURVA DE APRENDIZAJE

"If you know SQL, you know Drizzle"

CURVA DE APRENDIZAJE





TEMAS NO TÉCNICOS



Comunidad

Soporte activo de desarrolladores.

Ecosistema de plugins, foros y recursos compartidos.



Resolución de Bugs

Cómo aborda los problemas la tecnología.



Aprendizaje

Curva de adopción para nuevos miembros.

Facilidad de uso, documentación y requisitos previos requeridos.



COMUNIDAD

Descargas npm:

55.3M

Soporte activo y
ecosistema
consolidado.

**Estrellas en
Github:**

47.6k

Interés sostenido
de la comunidad
open source.

**Miembros en
Discord:**

10.3k

Espacio de soporte,
dudas y buenas
prácticas.

**Crecimiento
Agéntico:**

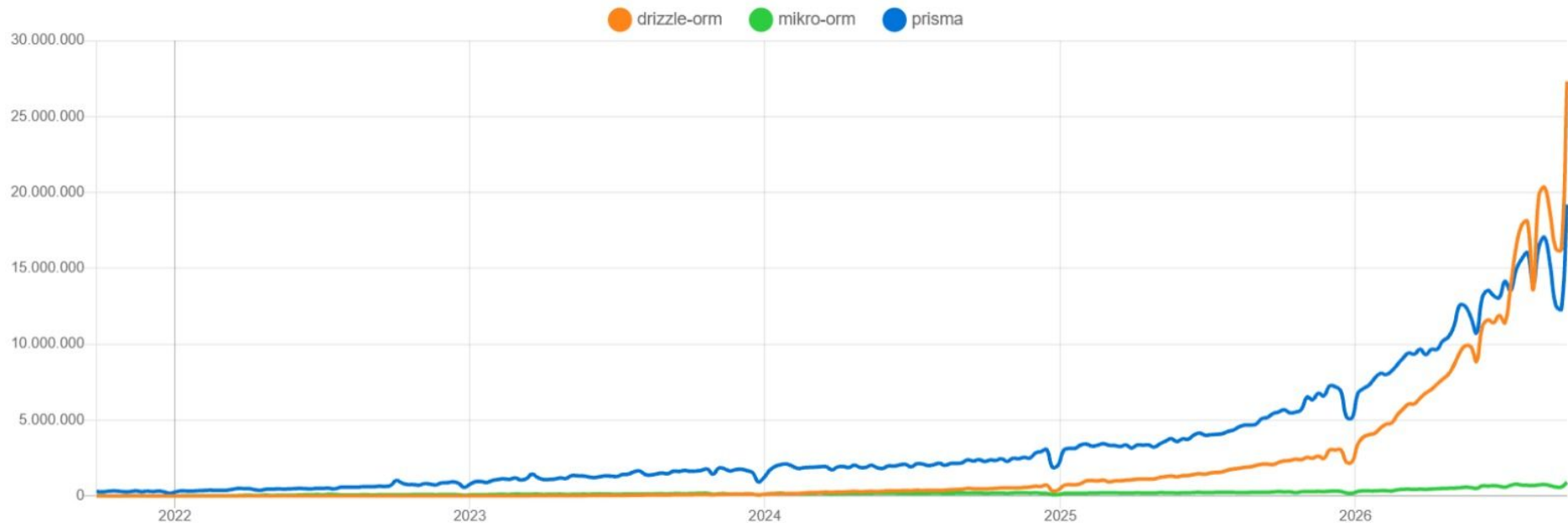
X4.4

Agentes de IA que
instalan y
scaffoldean
proyectos
automáticamente.



Gráfica

Downloads in past 5 Years ▾





RESOLUCIÓN DE BUGS

PROCESO Y CANALES

Colab entre la comunidad de usuarios y el equipo oficial de Prisma.

Canales de reporte: Discord para comunidad; Canales privados para seguridad.

Reproducción mínima: un template con comportamiento esperado vs. actual para aislar el fallo.

ROL DE LA COMUNIDAD

Tres pasos fundamentales al hallar un bug:

1. Aislamiento

2. Workarounds

3. Reporte

ROL DEL EQUIPO

1. Triage

2. Diagnóstico

3. Corrección

4. Seguimiento

Ciclo de vida del Bug: Reporte GitHub → Triage → Diagnóstico → Corrección → Pruebas → Release



CURVA DE APRENDIZAJE

Escalabilidad

Simple para empezar, no necesita SQL previo. Pero con features avanzadas de motores específicos (stored procedures, optimizaciones de Postgres), la abstracción juega en contra.

DSL

Aprender la sintaxis propia de `schema.prisma` tiene un costo — a cambio de mayor simplicidad general.

En general

Abstrae la complejidad de las bases relacionales. Ideal para devs Frontend/Full Stack que recién llegan al Backend.

Factores Técnicos

Veamos rápidamente un ejemplo

Supongamos que queremos hacer la siguiente consulta:

"Mostrar los usuarios cuya edad sea mayor a 18, ordenados por nombre de manera ascendente."

SQL

```
SELECT *  
FROM users  
WHERE age > 18  
ORDER BY name ASC;
```

DRIZZLE

```
const users = await db  
  .select()  
  .from(users)  
  .where(gt(users.age, 18))  
  .orderBy(asc(users.name));
```

SQL

```
SELECT *  
FROM users  
WHERE age > 18  
ORDER BY name ASC;
```

PRISMA

```
const users = await  
prisma.user.findMany({  
  where: { age: { gt: 18 } },  
  orderBy: { name: 'asc' }  
});
```



MODELO DECLARATIVO

SQL PURO

Gestión manual.

Paso por Paso.

PRISMA

Modelo Declarativo.

Forma Final.

ARCHIVO `schema.prisma`

- Fuente de Verdad.
- Evita la dispersión en la definición de los datos.
- Legibilidad.
- Separación de responsabilidades.

EJEMPLO CÓDIGO:

```
model User {  
  id Int @id @default(autoincrement())  
  email String @unique  
  posts Post [  
  ]  
}
```



PRISMA CLIENT

Se define el modelo en `schema.prisma` → Se ejecuta `prisma generate`
→ Prisma produce el código del cliente → Se Importa y usa el cliente en la aplicación

Produce código

TypeScript nuevo,
adaptado exactamente
a los modelos, campos
y relaciones definidos
en el `schema.prisma`.

TYPE SAFETY

Previene errores en
tiempo de compilación y
ofrece autocompletado
en el editor de código.

EJEMPLO CÓDIGO:

```
import { PrismaClient } from '@prisma/client'

const prisma = new PrismaClient()

const user = await prisma.user.findUnique({
  where: { email: "juan@mail.com" }
})
```



PRISMA MIGRATE

Al modificar el `schema.prisma`:

1. Compara el estado actual del schema con la base de datos.
2. Genera un archivo SQL de migración versionado.
3. Lo aplica.

1. Reproducibilidad.
2. Historial auditable.
3. Rollback

controlado

El archivo contiene el SQL explícito.

Prisma aplica automáticamente esa migración a la BD

EJEMPLO CÓDIGO:

```
model User {  
  id Int @id  
    @default(autoincrement())  
  email String @unique  
  name String? // campo nuevo  
  posts Post [  
  ]  
}
```



MANEJO DE RELACIONES

Reconstruye
automáticamente la
estructura de objetos
anidados.

Filtrado de relaciones

Se pueden cargar
relaciones anidadas,
filtrarlas y ordenarlas.

EJEMPLO CÓDIGO:

```
const user = await  
prisma.user.findUnique({  
  where: { id: 1 },  
  include: { posts: true }  
})
```



COMPATIBILIDAD MULTI-BASE DE DATOS

- PostgreSQL
- MySQL
- MariaDB
- SQLite
- SQL Server
- CockroachDB
- MongoDB como motor NoSQL

Configuración:

Se define en un solo bloque (datasource) dentro del schema.

EJEMPLO CÓDIGO:

```
datasource db {  
    provider = "postgresql"  
    url = env("DATABASE_URL")  
}
```



QUERY ENGINE

Antes

Query Engine escrito en rust que corría como proceso independiente.

- Overhead.
- Peso Grande (14 MB).
- Peor rendimiento que el driver nativo.

Ahora

- TypeScript puro.
- Peso reducido (100 KB).
- 90% del rendimiento del driver nativo.



Raw SQL

Es posible ejecutar SQL crudo
en caso de ser necesario.

Por ejemplo, al necesitar una
función específica de un motor.

```
const result = await prisma.$queryRaw  
`SELECT * FROM "User" WHERE age > ${18}`;  
  
await prisma.$executeRaw  
`UPDATE "User" SET activo = true  
WHERE "lastLogin" > ${fecha}`;
```

Parametrización automática.



Compatibilidad Serverless / Edge

Prisma Accelerate

Servicio gestionado por Prisma.

Connection pooling externo.

Caché de queries.

Se discontinúa el 1/12/2026 — el pooling pasa a integrarse directamente en Prisma Postgres.

Driver Adapters

Prisma permite conectar el cliente directamente a drivers nativos específicos de cada motor.

- **Reduce el tamaño del bundle final**
- **Permite aprovechar drivers ya optimizados para el entorno serverless.**

TEMAS TÉCNICOS

- **Conexión mediante drivers**

Drizzle se comunica con la base de datos utilizando drivers existentes, como node-postgres, postgres.js o mysql2.

No necesita un motor de consultas independiente entre la aplicación y el driver.

Esto lo hace más **liviano**, aparte de tener un bajo overhead.

TEMAS TÉCNICOS

- **Arquitectura Headless**

Drizzle no impone una arquitectura específica al proyecto. Puede integrarse en diferentes frameworks y runtimes sin obligar al desarrollador a estructurar la aplicación alrededor del ORM.

Esto lo hace muy **flexible**.

TEMAS TÉCNICOS

- **Dos APIs para realizar consultas**

En Drizzle se puede escribir una consulta con la SQL-like API (pensando en SQL y tablas) y con la Relational Query API (pensando en relaciones)

Además, Drizzle permite incorporar SQL personalizado dentro de sus consultas hechas con su API.

Veamos rápidamente un ejemplo

Supongamos que queremos hacer la siguiente consulta:

"Traeme los usuarios junto con sus publicaciones."

SQL-LIKE API

```
const result = await db
  .select()
  .from(users)
  .leftJoin(posts, eq(users.id, posts.userId));
```

RELATIONAL QUERY API

```
const result = await db.query.users.findMany({  
  with: {  
    posts: true  
  }  
});
```

RELATIONAL API + SQL PERSONALIZADO

```
const result = await db.query.users.findMany({
  columns: {
    id: true,
    name: true
  },
  with: {
    posts: {
      extras: {
        titleLength:
          sql<number>`length(${posts.title})`.as("title_length")
      }
    }
  }
});
```

Mercado Laboral

PARTE LABORAL

Drizzle ORM es una tecnología con una demanda creciente, aunque todavía no alcanza el nivel de adopción de Prisma o TypeORM.

PARTE LABORAL

Su uso está aumentando en proyectos modernos desarrollados con TypeScript, Next.js, Hono y arquitecturas serverless.

Se puede encontrar en Startups tecnológicas. Empresas que desarrollan aplicaciones Full Stack con Next.js. Equipos que priorizan el rendimiento y aplicaciones livianas.

PARTE LABORAL

Sin embargo, es poco común encontrar búsquedas que pidan "Drizzle ORM" como requisito excluyente.

Por lo que, conocer Drizzle representa un **valor agregado**, y **no** un **requisito excluyente**.



Adopción en el ecosistema productivo

Adopción

- **55.3M** descargas mensuales en npm (**julio 2026**).
- **Septiembre 2026:** datos de *npmtrends* muestran a Drizzle superando a Prisma en descargas semanales .

Oferta Laboral

2025 → 2026: las publicaciones laborales que exigían Prisma bajaron de **~80% a ~55%**.

Drizzle pasó de casi no ser mencionado a aparecer en **~30%** de esas mismas publicaciones en **poco más de un año**.

fuentes: tracking de *jsgurujobs.com*.



Startups ~~vs.~~ & Enterprise

Startups

- **Muy usado** por su experiencia de desarrollo.
- **Rápida iteración** con equipos chicos.

Enterprise

- **Ofrece un plan Enterprise** con soporte, SLAs (Service Level Agreement) y acompañamiento en seguridad y cumplimiento.
- **Herramientas como Prisma Accelerate** permiten manejar altas demandas en entornos serverless.

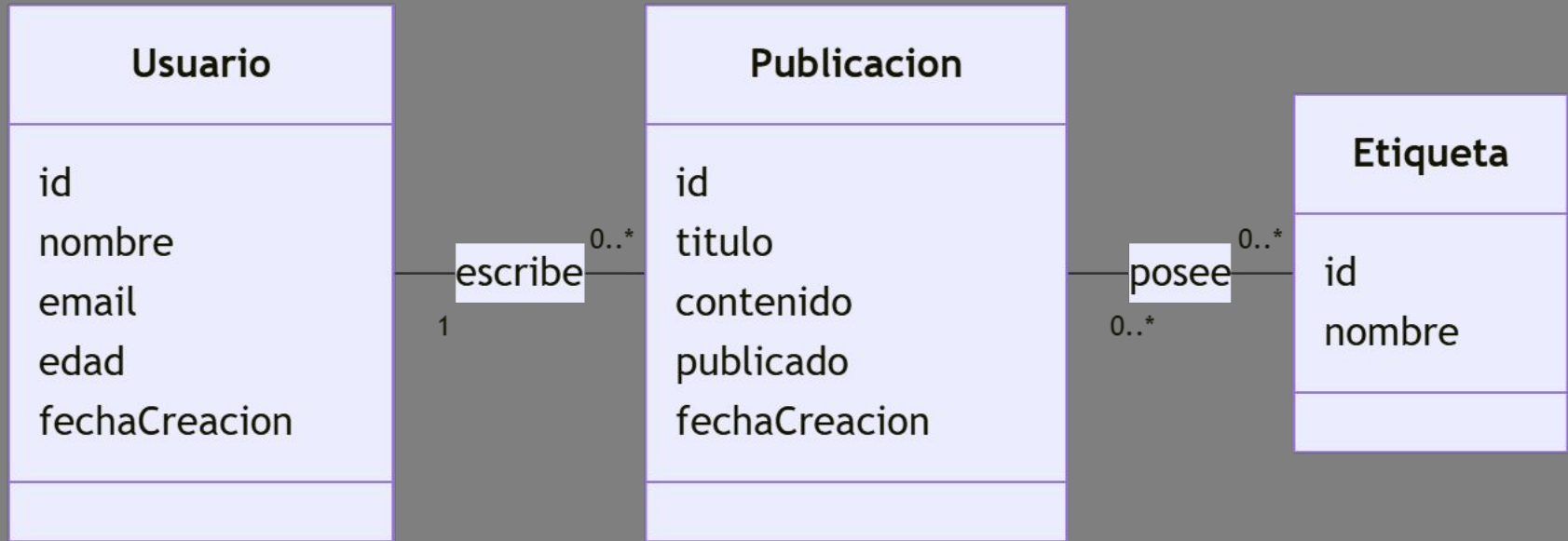


Rol como estándar

- Es una opción ampliamente integrada con guías oficiales para [Next.js](#), [NestJS](#), [Astro](#), Nuxt, SvelteKit, TanStack Start, Hono y Elysia.
- La integración con NestJS está bien documentada, pero no es automática. Existe una librería de la comunidad (nestjs-prisma) que automatiza ese armado, pero no es oficial.
- Para [Next.js](#) cuenta con starters, plantillas pre armadas que se pueden descargar y utilizar.

Implementación y demostración práctica (PoC)

DOMINIO



OPERACIONES

01

Modelado

**Definición
de Esquema**

02

Escritura

**Registro de
Datos**

(Registrar Usuario)

03

Lectura

**Consulta con
filtros**

**(Buscar publicaciones con
etiqueta “meme” y autor
mayor a 18 años)**

OPERACIONES

Configuración inicial

DRIZZLE

```
import { drizzle } from "drizzle-orm/mysql2";  
import { relations, and, eq, gt } from "drizzle-orm";  
import mysql from "mysql2/promise";
```

```
import { mysqlTable, int, varchar, text,  
  boolean, timestamp, primaryKey, } from "drizzle-orm/mysql-core";
```

DRIZZLE

```
const connection = mysql.createPool({  
  host: "localhost",  
  user: "root",  
  password: "...", //Contraseña de cada usuario  
  database: "prueba_drizzle",  
});
```

DRIZZLE

```
const schema = {  
  usuario, publicacion,  
  etiqueta, publicacionEtiqueta,  
  usuarioRelations, publicacionRelations,  
  etiquetaRelations, publicacionEtiquetaRelations,  
};
```

DRIZZLE

```
const db = drizzle(connection,  
  {schema,  
    mode: "default",  
  },  
);
```

Base del Script

```
1  import "dotenv/config";
2  import { PrismaClient } from "../src/generated/prisma/client.js";
3
4  const prisma = new PrismaClient();
5
6  const titulo = (t: string) =>
7    console.log(`\n${"=".repeat(50)}\n${t}\n${"=".repeat(50)}`);
8
9  async function main() {
10    // Limpia datos previos
11    await prisma.publicacion.deleteMany();
12    await prisma.etiqueta.deleteMany();
13    await prisma.usuario.deleteMany();
14  }
```

Base del schema.prisma

```
Generate
1 generator client {
2   provider = "prisma-client"
3   output   = "../src/generated/prisma"
4 }
5
6 datasource db {
7   provider = "mysql"
8   url       = env("DATABASE_URL")
9 }
10
```

schema.prisma completo

```
11  model Usuario {
12    id          Int          @id @default(autoincrement())
13    nombre      String?
14    email       String?      @unique
15    edad        Int?
16    fechaCreacion DateTime    @default(now())
17    publicaciones Publicacion[]
18  }
19
20  model Publicacion {
21    id          Int          @id @default(autoincrement())
22    titulo      String
23    contenido    String?
24    publicado    Boolean      @default(false)
25    fechaCreacion DateTime    @default(now())
26    autor        Usuario      @relation(fields: [autorId], references: [id])
27    autorId      Int
28    etiquetas    Etiqueta[]
29  }
30
31  model Etiqueta {
32    id          Int          @id @default(autoincrement())
33    nombre      String        @unique
34    publicaciones Publicacion[]
35  }
36
```

OPERACIONES

Modelado

SQL-LIKE API

```
export const usuario = mysqlTable("usuario", {  
  id: int("id").autoincrement().primaryKey(),  
  nombre: varchar("nombre", { length: 100 }).notNull(),  
  email: varchar("email", { length: 255 }).notNull().unique(),  
  edad: int("edad").notNull(),  
  fechaCreacion: timestamp("fecha_creacion").defaultNow().notNull(),  
});
```

SQL-LIKE API

```
export const publicacion = mysqlTable("publicacion", {
  id: int("id").autoincrement().primaryKey(),
  titulo: varchar("titulo", { length: 255 }).notNull(),
  contenido: text("contenido").notNull(),
  publicado: boolean("publicado").default(false).notNull(),
  fechaCreacion: timestamp("fecha_creacion").defaultNow().notNull(),

  usuarioId: int("usuario_id")
    .notNull()
    .references(() => usuario.id),
});
```

SQL-LIKE API

```
export const etiqueta = mysqlTable("etiqueta", {  
  id: int("id").autoincrement().primaryKey(),  
  nombre: varchar("nombre", { length: 100 }).notNull(),  
});
```

SQL-LIKE API

```
export const publicacionEtiqueta = mysqlTable(  
  "publicacion_etiqueta",  
  {  
    publicacionId: int("publicacion_id")  
      .notNull()  
      .references(() => publicacion.id),  
  
    etiquetaId: int("etiqueta_id")  
      .notNull()  
      .references(() => etiqueta.id),  
  },  
  (table) => [  
    primaryKey({  
      columns: [table.publicacionId, table.etiquetaId],  
    })  
  ]  
)
```

RELATIONAL QUERY API

```
export const usuarioRelations = relations(usuario,  
  ({ many }) => ({  
    publicaciones: many(publicacion),  
  }));
```

RELATIONAL QUERY API

```
export const publicacionRelations = relations({
  publicacion,
  ({ one, many }) => ({
    autor: one(usuario, {
      fields: [publicacion.usuarioId],
      references: [usuario.id],
    }),
    publicacionesEtiquetas: many(publicacionEtiqueta),
  })
})
```

RELATIONAL QUERY API

```
export const etiquetaRelations = relations(etiqueta,  
  ({ many }) => ({  
    publicacionesEtiquetas: many(publicacionEtiqueta),
```

RELATIONAL QUERY API

```
export const publicacionEtiquetaRelations = relations(  
  publicacionEtiqueta,  
  ({ one }) => ({  
    publicacion: one(publicacion, {  
      fields: [publicacionEtiqueta.publicacionId],  
      references: [publicacion.id],  
    }),  
    etiqueta: one(etiqueta, {  
      fields: [publicacionEtiqueta.etiquetaId],  
      references: [etiqueta.id],  
    })  
  })  
);
```

MODELADO

```
15 // ----- 01. MODELADO -----  
16 titulo("01 · MODELADO: esquema aplicado en la base");  
17 ✓ const tablas = await prisma.$queryRaw<{ tabla: string }[]>`  
18     SELECT TABLE_NAME AS tabla  
19     FROM information_schema.tables  
20     WHERE table_schema = DATABASE() AND TABLE_NAME NOT LIKE '\\_%'  
21 `;  
22 console.log("Tablas en la base:", tablas.map((t) => t.tabla));  
23
```

OPERACIONES

**Escritura
Registro de Datos
(Registrar Usuario)**

SQL-LIKE API

```
async function main() {  
  await db.insert(usuario).values({  
    nombre: "Juan",  
    email: "juan@gmail.com",  
    edad: 20,  
  });  
}
```

Carga de Datos de Prueba

```
const ana = await prisma.usuario.create({
  data: {
    nombre: "Ana",
    email: "ana@ejemplo.com",
    edad: 28,
    publicaciones: {
      create: {
        titulo: "Mi primer meme",
        contenido: "jaja",
        publicado: true,
        etiquetas: {
          connectOrCreate: {
            where: { nombre: "meme" },
            create: { nombre: "meme" },
          },
        },
      },
    },
  },
});
console.log("Usuario creado:", ana);
```

```
await prisma.usuario.create({
  data: {
    nombre: "Leo",
    email: "leo@ejemplo.com",
    edad: 15,
    publicaciones: {
      create: {
        titulo: "Meme de Leo",
        publicado: true,
        etiquetas: {
          connectOrCreate: {
            where: { nombre: "meme" },
            create: { nombre: "meme" },
          },
        },
      },
    },
  },
});
```

```
await prisma.usuario.create({
  data: {
    nombre: "Marta",
    email: "marta@ejemplo.com",
    edad: 35,
    publicaciones: {
      create: {
        titulo: "Receta de empanadas",
        publicado: true,
        etiquetas: {
          connectOrCreate: {
            where: { nombre: "cocina" },
            create: { nombre: "cocina" },
          },
        },
      },
    },
  },
});
```

Demostración de Validación

```
try {  
  await prisma.usuario.create({  
    data: { nombre: "Otra Ana", email: "ana@ejemplo.com", edad: 20 },  
  });  
} catch (e: any) {  
  console.log(`Email duplicado rechazado (código ${e.code})`);  
}
```

OPERACIONES

Lectura

Búsqueda por Criterio

(Buscar publicaciones con etiqueta
“meme” y autor mayor a 18 años)

DRIZZLE

**Primero hacemos una carga
de datos para poder hacer la
consulta**

SQL-LIKE API

```
//Pedro - 17 años
await db.insert(usuario).values({
  nombre: "Pedro",
  email: "pedro@gmail.com",
  edad: 17,
});

//Etiquetas
await db.insert(etiqueta).values([
  { nombre: "meme" },
  { nombre: "programacion" },
]);
```

SQL-LIKE API

```
//Publicaciones
await db.insert(publicacion).values([
  {
    titulo: "Meme de SQL",
    contenido: "Cuando te olvidás el WHERE",
    publicado: true,
    usuarioId: 1, // Juan
```

SQL-LIKE API

```
titulo: "Aprendiendo Drizzle",  
contenido: "Probando Drizzle ORM",  
publicado: true,  
usuarioId: 1, // Juan
```

SQL-LIKE API

```
{  
  titulo: "Otro meme",  
  contenido: "Un meme publicado por Pedro",  
  publicado: true,  
  usuarioId: 2, // Pedro
```

SQL-LIKE API

```
//Relación Publicacion con Etiqueta
await db.insert(publicacionEtiqueta).values([
  {
    publicacionId: 1,
    etiquetaId: 1, // Meme de SQL -> meme
  },
  {
    publicacionId: 2,
    etiquetaId: 2, // Aprendiendo Drizzle -> programacion
  },
  {
    publicacionId: 3,
    etiquetaId: 1, // Otro meme -> meme
  },
]);
```

SQL-LIKE API

```
const resultado = await db
  .select({
    publicacion: publicacion.titulo, autor: usuario.nombre,
    edad: usuario.edad, etiqueta: etiqueta.nombre,})
  .from(publicacion)
  .innerJoin(usuario,
    eq(publicacion.usuarioId, usuario.id))
  .innerJoin(publicacionEtiqueta,
    eq(publicacion.id, publicacionEtiqueta.publicacionId))
  .innerJoin(etiqueta,
    eq(publicacionEtiqueta.etiquetaId, etiqueta.id))
  .where(and(eq(etiqueta.nombre, "meme"),
    gt(usuario.edad, 18)));
console.log(resultado);
```

RELATIONAL QUERY API

```
const publicaciones = await db.query.publicacion.findMany({  
  with: {  
    autor: true,  
    publicacionesEtiquetas: {  
      with: {  
        etiqueta: true,  

```

RELATIONAL QUERY API

```
const resultadoRelational = publicaciones
  .filter(
    (pub) =>
      pub.autor.edad > 18 &&
      pub.publicacionesEtiquetas.some(
        (pe) => pe.etiqueta.nombre === "meme"
      )
  )
  .map((pub) => ({
    publicacion: pub.titulo,
    autor: pub.autor.nombre,
    edad: pub.autor.edad,
    etiqueta: "meme",
  }));

console.log(resultadoRelational);
```

Lectura

```
titulo('03 · LECTURA: publicaciones con etiqueta "meme" y autor > 18');

const resultado = await prisma.publicacion.findMany({
  where: {
    etiquetas: { some: { nombre: "meme" } },
    autor: { edad: { gt: 18 } },
  },
  include: { autor: true, etiquetas: true },
});

console.table(
  resultado.map((p) => ({
    publicacion: p.titulo,
    autor: p.autor.nombre,
    edad: p.autor.edad,
    etiquetas: p.etiquetas.map((e) => e.nombre).join(", "),
  }))
);
```

Lectura

```
main()  
  .catch((e) => console.error(e))  
  .finally(() => prisma.$disconnect());
```

OPERACIONES

Salidas del ejercicio:

Lectura

Búsqueda por Criterio

**(Buscar publicaciones con etiqueta
“meme” y autor mayor a 18 años)**

SQL-LIKE API

Consulta con SQL like api

```
[  
  {  
    publicacion: 'Meme de SQL',  
    autor: 'Juan',  
    edad: 20,  
    etiqueta: 'meme'  
  }  
]
```

RELATIONAL QUERY API

Consulta con relational query api

```
[  
  {  
    publicacion: 'Meme de SQL',  
    autor: 'Juan',  
    edad: 20,  
    etiqueta: 'meme'  
  }  
]
```

Salida

```
=====
01 · MODELADO: esquema aplicado en la base
=====
Tablas en la base: [ 'etiqueta', 'publicacion', 'usuario' ]

=====
02 · ESCRITURA: registrar usuarios
=====
Usuario creado: {
  id: 9,
  nombre: 'Ana',
  email: 'ana@ejemplo.com',
  edad: 28,
  fechaCreacion: 2026-10-05T14:29:39.254Z
}
Total de usuarios: 3
```

Salida

```
=====
03 · LECTURA: publicaciones con etiqueta "meme" y autor > 18
=====
```

(index)	publicacion	autor	edad	etiquetas
0	'Mi primer meme'	'Ana'	28	'meme'

Esperado: solo la publicación de Ana (Leo es menor, Marta no tiene 'meme').

INTERFACES GRÁFICAS

**Vemos las tablas
desde las interfaces gráficas
de cada OrM**

DRIZZLE STUDIO: USUARIO

☐	id int ↕	nombre varchar(100) ↕	email varchar(255) ↕	edad int ↕	fecha_creacion timestamp ↕	publicacions
☐	1	Juan	juan@gmail.com	20	2026-10-03 14:54:31	publicacions
☐	2	Pedro	pedro@gmail.com	17	2026-10-03 14:58:13	publicacions

DRIZZLE STUDIO: ETIQUETA

☐	<code>id int</code>	↕	<code>nombre varchar(100)</code>	↕	<code>publicacion_etiquetas</code>
☐	<code>1</code>		<code>meme</code>		<code>publicacion_etiquetas</code>
☐	<code>2</code>		<code>programacion</code>		<code>publicacion_etiquetas</code>

DRIZZLE STUDIO: PUBLICACIÓN

☐	id int	↕	titulo varchar(255)	↕	contenido text	↕	publicado tinyint(1)	↕	fecha_creacion timestamp	↕	usuario_id int	↕	usuario	publicacion_etiquetas
☐	1		Meme de SQL		Cuando te olvidás el W...		1		2026-10-03 14:58:13		1		usuario	publicacion_etiquetas
☐	2		Aprendiendo Drizzle		Probando Drizzle ORM		1		2026-10-03 14:58:13		1		usuario	publicacion_etiquetas
☐	3		Otro meme		Un meme publicado por ...		1		2026-10-03 14:58:13		2		usuario	publicacion_etiquetas

DRIZZLE STUDIO: PUBLICACION_ETIQUETA

☐	publicacion_id int	☐	etiqueta_id int	☐	etiqueta	☐	publicacion
☐	1	☐	1	☐	etiqueta	☐	publicacion
☐	2	☐	2	☐	etiqueta	☐	publicacion
☐	3	☐	1	☐	etiqueta	☐	publicacion

Prisma Studio

	Publicacion ×	Usuario ×	Etiqueta ×	Close all				
	Filters	None	Fields	All	Showing	3 of 3	Add record	
	id #	titulo A	contenido A?	publicado ➔	fechaCreacion 📅	autor {}	autorId #	
	7	Mi primer meme	jaja	true	2026-10-05T14:29:39.254Z	Usuario	9	
	8	Meme de Leo	<i>null</i>	true	2026-10-05T14:29:39.273Z	Usuario	10	
	9	Receta de empanadas	<i>null</i>	true	2026-10-05T14:29:39.276Z	Usuario	11	

Prisma Studio

Publicacion

Usuario

Etiqueta

Close all

Filters

None

Fields

All

Showing

3 of 3

Add record

id

#

7

8

9

id

Int

titulo

String

contenido

String?

publicado

Boolean

fechaCreacion

DateTime

autor

Usuario

autorId

Int

etiquetas

Etiqueta[]

conten

jaja

null

das

null

Prisma Studio

	Publicacion ×	Usuario ×	Etiqueta ×	Close all		
	Filters 1	Fields All	Showing 3 of 3	Add record		
where	id	▼	equals ▼	enter value...	×	
+ Add a new filter		Clear all				
☐	id #	nombre A?	email A?	edad #?	fechaCreacion 📅	publicaciones []
☐	9	Ana	ana@ejemplo.com	28	2026-10-05T14:29:39.254Z	1 Publicacion
☐	10	Leo	leo@ejemplo.com	15	2026-10-05T14:29:39.273Z	1 Publicacion
☐	11	Marta	marta@ejemplo.com	35	2026-10-05T14:29:39.276Z	1 Publicacion

Prisma Studio

Publicacion

×

Usuario

×

Etiqueta

×

Close all

Filters

None

Fields

All

Showing

2 of 2

Add record

	id #	nombre A	publicaciones []
	5	meme	2 Publicacion
	6	cocina	1 Publicacion

Conclusiones

Conclusiones



Tendencia de Mercado

Conclusiones



Tendencia de Mercado



**Curva de aprendizaje y
escalabilidad a largo plazo**

Conclusiones



Tendencia de Mercado



Curva de aprendizaje y
escalabilidad a largo plazo



Tipo de proyecto en el que se
destaca cada uno

Conclusiones



Tendencia de Mercado



Curva de aprendizaje y
escalabilidad a largo plazo



Tipo de proyecto en el que se
destaca cada uno



Mercado laboral,
¿cuál conviene más?

POC ORM



PRISMA

53742 - Nicolás Bolzico

53952 - Martín Cabrera

52285 - Agustín Gregoret



DRIZZLE

54326- Roque Doino

54689- Juan Pablo Villa